Build your app from the command line

You can execute all the build tasks available to your Android project using the Gradle wrapper (https://docs.gradle.org/current/userguide/gradle_wrapper.html) command line tool. It's available as a batch file for Windows (`gradlew.bat`) and a shell script for Linux and Mac (`gradlew.sh`), and it's accessible from the root of each project you create with Android Studio.

To run a task with the wrapper, use one of the following commands from a Terminal window (from Android Studio, select **View > Tool Windows > Terminal**):

- On Windows Command Shell:

```
gradlew task-name
```

- On Mac, Linux, or Windows PowerShell:

```
./gradlew task-name
```

To see a list of all available build tasks for your project, execute `tasks`:

```
gradlew tasks
```

The rest of this page describes the basics to build and run your app with the Gradle wrapper. For more information about how to set up your Android build, see Configure your build (/studio/build).

If you'd prefer to use the Android Studio tools instead of the command line tools, see Build and run your app (/studio/run).

About build types

By default, there are two build types available for every Android app: one for debugging your app—the *debug* build—and one for releasing your app to users—the *release* build. The resulting output from each build must be signed with a certificate before you can deploy your app to a device. The debug build is automatically signed with a debug key provided by the SDK tools (it's insecure and you cannot publish with it to the Google Play Store), and the release build must be signed with your own private key.

If you want to build your app for release, it's important that you also sign your app (#sign_cmdline) with the appropriate signing key. If you're just getting started, however, you can quickly run your apps on an emulator or a connected device by building a debug APK (#DebugMode).

You can also define a custom build type in your `build.gradle.kts` file and configure it to be signed as a debug build by including `debuggable true`. For more information, see Configure Build Variants (/studio/build/build-variants).

Build and deploy an APK

Although building an app bundle (#build_bundle) is the best way to package your app and upload it to the Play Console, building an APK is better suited for when you want quickly test a debug build or share your app as a deployable artifact with others.

Build a debug APK

For immediate app testing and debugging, you can build a debug APK. The debug APK is signed with a debug key provided by the SDK tools and allows debugging through `adb` (/studio/command-line/adb).

To build a debug APK, open a command line and navigate to the root of your project directory. To initiate a debug build, invoke the `assembleDebug` task:

```
gradlew assembleDebug
```

This creates an APK named *`module_name`*`-debug.apk` in *`project_name`*`/`*`module_name`*`/build/outputs/apk/`. The file is already signed with the

debug key and aligned with `zipalign` (/tools/help/zipalign), so you can immediately install it on a device.

Or to build the APK and immediately install it on a running emulator or connected device, instead invoke `installDebug`:

```
gradlew installDebug
```

The "Debug" part in the above task names is just a camel-case version of the build variant (/studio/build/build-variants) name, so it can be replaced with whichever build type or variant you want to assemble or install. For example, if you have a "demo" product flavor, then you can build the debug version with the `assembleDemoDebug` task.

To see all the build and install tasks available for each variant (including uninstall tasks), run the `tasks` task.

Also see the section about how to run your app on the emulator (#RunningOnEmulator) and run your app on a device (#RunningOnDevice).

Build a release bundle or APK

When you're ready to release and distribute your app, you must build a release bundle or APK that is signed with your private key. For more information, go to the section about how to sign your app from the command line (#sign_cmdline).

Deploy your app to the emulator

To use the Android Emulator, you must create an Android Virtual Device (AVD) (/studio/run/managing-avds#createavd) using Android Studio.

Once you have an AVD, start the Android Emulator and install your app as follows:

1. In a command line, navigate to *`android_sdk`*`/tools/` and start the emulator by specifying your AVD:

   ```
   emulator -avd avd_name
   ```

If you're unsure of the AVD name, execute `emulator -list-avds`.

2. Now you can install your app using either one of the Gradle install tasks mentioned in the section about how to build a debug APK (#DebugMode) or the `adb` (/studio/command-line/adb) tool.

   If the APK is built using a developer preview SDK (if the `targetSdkVersion` is a letter instead of a number), you must include the `-t` option (/studio/command-line/adb#-t-option) with the `install` command to install a test APK.

   ```
   adb install path/to/your_app.apk
   ```

   All APKs you build are saved in `project_name/module_name/build/outputs/apk/`.

For more information, see Run Apps on the Android Emulator (/studio/run/emulator).

Deploy your app to a physical device

Before you can run your app on a device, you must enable **USB debugging** on your device. You can find the option under **Settings > Developer options**.

Note: On Android 4.2 and newer, **Developer options** is hidden by default. To make it available, go to **Settings > About phone** and tap **Build number** seven times. Return to the previous screen to find **Developer options**.

Once your device is set up and connected via USB, you can install your app using either the Gradle install tasks mentioned in the section about how to build a debug APK (#DebugMode) or the `adb` (/studio/command-line/adb) tool:

```
adb -d install path/to/your_app.apk
```

All APKs you build are saved in `project_name/module_name/build/outputs/apk/`.

For more information, see Run Apps on a Hardware Device (/studio/run/device).

Build an app bundle

Android App Bundles (/guide/app-bundle) include all your app's compiled code and resources, but defer APK generation and signing to Google Play. Unlike an APK, you can't deploy an app bundle directly to a device. So, if you want to quickly test or share an APK with someone else, you should instead build an APK (#build_apk).

The easiest way to build an app bundle is by using Android Studio (/studio/run#reference). However, if you need to build an app bundle from the command line, you can do so by using either Gradle or `bundletool`, as described in the sections below.

Build an app bundle with Gradle

If you'd rather generate an app bundle from the command line, run the `bundle`*`Variant`* Gradle task on your app's base module. For example, the following command builds an app bundle for the debug version of the base module:

```
./gradlew :base:bundleDebug
```

If you want to build a signed bundle for upload to the Play Console, you need to first configure the base module's `build.gradle.kts` file with your app's signing information. To learn more, go to the section about how to Configure Gradle to sign your app (#gradle_signing). You can then, for example, build the release version of your app, and Gradle automatically generates an app bundle and signs it with the signing information you provide in the `build.gradle.kts` file.

If you instead want to sign an app bundle as a separate step, you can use `jarsigner` (https://docs.oracle.com/javase/8/docs/technotes/tools/windows/jarsigner.html) to sign your app bundle from the command line. The command to build an app bundle is:

```
$ jarsigner -keystore pathToKeystore app-release.aab keyAlias
```

Note: You cannot use `apksigner` to sign your app bundle.

Build an app bundle using bundletool

`bundletool` is a command line tool that Android Studio, the Android Gradle plugin, and Google Play use to convert your app's compiled code and resources into app bundles, and generate deployable APKs from those bundles.

So, while it's useful to test app bundles with `bundletool` (/studio/command-line/bundletool) and locally recreate how Google Play generates APKs, you typically won't need to invoke `bundletool` to build the app bundle itself—you should instead use Android Studio or Gradle tasks, as described in previous sections.

However, if you don't want to use Android Studio or Gradle tasks to build bundles—for example, if you use a custom build toolchain—you can use `bundletool` from the command line to build an app bundle from pre-compiled code and resources. If you haven't already done so, download `bundletool` (https://github.com/google/bundletool/releases) from the GitHub repository.

This section describes how to package your app's compiled code and resources, and how to use `bundletool` from the command line to convert them into an Android App Bundle.

Generate the manifest and resources in proto format

`bundletool` requires certain information about your app project, such as the app's manifest and resources, to be in Google's Protocol Buffer format (https://developers.google.com/protocol-buffers/)—which is also known as "protobuf" and uses the `*.pb` file extension. Protobufs provide a language-neutral, platform-neutral, and extensible mechanism for serializing structured data—it's similar to XML, but smaller, faster, and simpler.

Download AAPT2

You can generate your app's manifest file and resource table in protobuf format using the latest version of AAPT2 from the Google Maven repository (/studio/build/dependencies#google-maven).

Caution: Do not use the version of AAPT2 that is included in the Android build tools package—that version of AAPT2 does not support `bundletool`.

To download AAPT2 from Google's Maven repository, proceed as follows:

1. Navigate to **com.android.tools.build > aapt2** in the repository index (https://maven.google.com/).
2. Copy the name of the latest version of AAPT2.
3. Insert the version name you copied into the following URL and specify your target operating system: https://dl.google.com/dl/android/maven2/com/android/tools/build/aapt2/***aapt2-version***/aapt2-***aapt2-version-[windows | linux | osx]***.jar

   For example, to download version 3.2.0-alpha18-4804415 for Windows, you would use: https://dl.google.com/dl/android/maven2/com/android/tools/build/aapt2/**3.2.0-alpha18-4804415**/aapt2-**3.2.0-alpha18-4804415-windows**.jar
4. Navigate to the URL in a browser—AAPT2 should begin downloading shortly.
5. Unpackage the JAR file you just downloaded.

Compile and link your app's resources

Use AAPT2 to compile your app's resources with the following command:

```
aapt2 compile \
project_root/module_root/src/main/res/drawable/Image1.png \
project_root/module_root/src/main/res/drawable/Image2.png \
-o compiled_resources/
```

Note: Although you can pass resource directories to AAPT2 using the `--dir` flag, doing so recompiles all files in the directory, regardless of how many files actually changed.

During the link phase, where AAPT2 links your various compiled resources into a single APK, instruct AAPT2 to convert your app's manifest and compiled resources into the protobuf format by including the `--proto-format` flag, as shown below:

```
aapt2 link --proto-format -o output.apk \
-I android_sdk/platforms/android_version/android.jar \
--manifest project_root/module_root/src/main/AndroidManifest.xml \
```

```
-R compiled_resources/*.flat \
--auto-add-overlay
```

Note: Alternatively, when specifying compiled resources with the `-R` flag, you can specify a text file that includes the absolute path for each of your compiled resources—with each path separated by a single space. You can then pass that text file to AAPT2 as follows: `aapt2 link ... -R @compiled_resources.txt`.

You can then extract content from the output APK, such as your app's `AndroidManifest.xml`, `resources.pb`, and other resource files—now in the protobuf format. You need these files when preparing the input `bundletool` requires to build your app bundle, as described in the following section.

Package pre-compiled code and resources

Before you use `bundletool` to generate an app bundle for your app, you must first provide ZIP files that each contain the compiled code and resources for a given app module. The content and organization of each module's ZIP file is very similar to that of the Android App Bundle format (/guide/app-bundle#aab_format). For example, you should create a `base.zip` file for your app's base module and organize its contents as follows:

File or directory	Description
`manifest/AndroidManifest.xml`	The module's manifest in protobuf format.
`dex/...`	A directory with one or more of your app's compiled DEX files. These files should be named as follows: `classes.dex`, `classes2.dex`, `classes3.dex`, etc.
`res/...`	Contains the module's resources in protobuf format for all device configurations. Subdirectories and files should be organized similar to that of a typical APK.
`root/...`, `assets/...`, and `lib/...`	These directories are identical to those described in the section about the Android app bundle format (/guide/app-bundle#aab_format).

`resources.pb`	Your app's resource table in protobuf format.

After you prepare the ZIP file for each module of your app, you can pass them to `bundletool` to build your app bundle, as described in the following section.

Build your app bundle using bundletool

To build your app bundle, you use the `bundletool build-bundle` command, as shown below:

```
bundletool build-bundle --modules=base.zip --output=mybundle.aab
```

Note: If you plan to publish the app bundle, you need to sign it using `jarsigner` (https://docs.oracle.com/javase/8/docs/technotes/tools/windows/jarsigner.html). You can not use `apksigner` to sign your app bundle.

The following table describes flags for the `build-bundle` command in more detail:

Flag	Description
`--modules=path-to-base.zip,path-to-module2.zip,path-to-module3.zip`	Specifies the list of module ZIP files `bundletool` should use to build your app bundle.
`--output=path-to-output.aab`	Specifies the path and filename for the output `*.aab` file.
`--config=path-to-BundleConfig.json`	Specifies the path to an optional configuration file you can use to customize the build process. To learn more, see the section about customizing downstream APK generation (#bundleconfig).
`--metadata-file=target-bundle-path:local-file-path`	Instructs `bundletool` to package an optional metadata file inside your app bundle. You can use this file to include data, such as ProGuard mappings or the complete list of your app's DEX files,

that may be useful to other steps in your toolchain or an app store.

target-bundle-path specifies a path relative to the root of the app bundle where you would like the metadata file to be packaged, and ***local-file-path*** specifies the path to the local metadata file itself.

Customize downstream APK generation

App bundles include a `BundleConfig.pb` file that provides metadata that app stores, such as Google Play, require when generating APKs from the bundle. Although `bundletool` creates this file for you, you can configure some aspects of the metadata in a `BundleConfig.json` file and pass it to the `bundletool build-bundle` command—`bundletool` later converts and merges this file with the protobuf version included in each app bundle.

Note: To learn more about how the JSON maps to the protobuf format, read JSON Mapping (https://developers.google.com/protocol-buffers/docs/proto3#json). However, that information is more advanced than what is required for this page.

For example, you can control which categories of configuration APKs to enable or disable. The following example of an `BundleConfig.json` file disables configuration APKs that each target a different language (that is, resources for all languages are included in their respective base or feature APKs):

```
{
  "optimizations": {
    "splitsConfig": {
      "splitDimension": [{
        "value": "LANGUAGE",
        "negate": true
      }]
    }
  }
}
```

In your `BundleConfig.json` file, you can also specify which file types to leave uncompressed when packaging APKs using glob patterns (https://docs.oracle.com/javase/tutorial/essential/io/fileOps.html#glob), as follows:

```
{
  "compression": {
    "uncompressedGlob": ["res/raw/**", "assets/**.uncompressed"]
  }
}
```

Keep in mind, by default, `bundletool` does not compress your app's native libraries (on Android 6.0 or higher) and resource table (`resources.arsc`). For a full description of what you can configure in your `BundleConfig.json`, inspect the `bundletool config.proto` file (https://github.com/google/bundletool/blob/master/src/main/proto/config.proto), which is written using Proto3 (https://developers.google.com/protocol-buffers/docs/proto3) syntax.

Deploy your app from an app bundle

If you've built and signed an app bundle, use `bundletool` (/studio/command-line/bundletool) to generate APKs and deploy them to a device.

Sign your app from command line

You do not need Android Studio to sign your app. You can sign your app from the command line, using `apksigner` for APKs or `jarsigner` for app bundles, or configure Gradle to sign it for you during the build. Either way, you need to first generate a private key using `keytool` (https://docs.oracle.com/javase/8/docs/technotes/tools/unix/keytool.html), as shown below:

```
$ keytool -genkey -v -keystore my-release-key.jks -keyalg RSA -keysize 2048 -
```

The example above prompts you for passwords for the keystore and key, and for the "Distinguished Name" fields for your key. It then generates the keystore as a file called

`my-release-key.jks`, saving it in the current directory (you can move it wherever you'd like). The keystore contains a single key that is valid for 10,000 days.

Now you can sign you APK or app bundle manually, or configure Gradle to sign your app during the build process, as described in the sections below.

Sign your app manually from the command line

If you want to sign an app bundle from the command line, you can use `jarsigner` (https://docs.oracle.com/javase/8/docs/technotes/tools/windows/jarsigner.html). If instead you want to sign an APK, you need to use `zipalign` and `apksigner` as described below.

1. Open a command line—from Android Studio, select **View > Tool Windows > Terminal**—and navigate to the directory where your unsigned APK is located.
2. Align the unsigned APK using `zipalign` (/studio/command-line/zipalign):

   ```
   $ zipalign -v -p 4 my-app-unsigned.apk my-app-unsigned-aligned.apk
   ```

   `zipalign` ensures that all uncompressed data starts with a particular byte alignment relative to the start of the file, which may reduce the amount of RAM consumed by an app.
3. Sign your APK with your private key using `apksigner` (/studio/command-line/apksigner):

   ```
   $ apksigner sign --ks my-release-key.jks --out my-app-release.apk my-app-
   ```

   This example outputs the signed APK at `my-app-release.apk` after signing it with a private key and certificate that are stored in a single KeyStore file: `my-release-key.jks`.

   The `apksigner` tool supports other signing options, including signing an APK file using separate private key and certificate files, and signing an APK using multiple signers. For more details, see the `apksigner` (/studio/command-line/apksigner) reference.

★ **Note:** To use the `apksigner` tool, you must have revision 24.0.3 or higher of the Android SDK Build Tools installed. You can update this package using the SDK Manager (/studio/intro/update#sdk-manager).

4. Verify that your APK is signed:

```
$ apksigner verify my-app-release.apk
```

Configure Gradle to sign your app

Open the module-level `build.gradle.kts` file and add the `signingConfigs {}` block with entries for `storeFile`, `storePassword`, `keyAlias` and `keyPassword`, and then pass that object to the `signingConfig` property in your build type. For example:

Kotlin build.gradle.kts | Groovy build.gradle

```
android {
    ...
    defaultConfig { ... }
    signingConfigs {
        create("release") {
            // You need to specify either an absolute path or include th
            // keystore file in the same directory as the build.gradle f
            storeFile = file("my-release-key.jks")
            storePassword = "password"
            keyAlias = "my-alias"
            keyPassword = "password"
        }
    }
    buildTypes {
        getByName("release") {
            signingConfig = signingConfigs.getByName("release")
            ...
        }
    }
}
```

Note: In this case, the keystore and key password are visible directly in the **`build.gradle`** file. For improved security, you should remove the signing information from your build file (/studio/publish/app-signing#secure-shared-keystore).

Now, when you build your app by invoking a Gradle task, Gradle signs your app (and runs zipalign) for you.

Additionally, because you've configured the release build with your signing key, the "install" task is available for that build type. So you can build, align, sign, and install the release APK on an emulator or device all with the `installRelease` task.

An app signed with your private key is ready for distribution, but you should first read more about how to publish your app (/studio/publish) and review the Google Play launch checklist (/distribute/tools/launch-checklist).